

CLAIRE

Claude Loop for Adaptive Intelligence from Real-world Evidence

A weekly AI self-optimization pipeline driven by community signal.

PIPELINE STAGES

5 Stages

AI MODELS

3 Models

OUTPUT

Weekly Digest

CADENCE

Fully Automated

Contents

1. What is CLAIRE?
2. The Problem It Solves
3. Pipeline Architecture
4. CLAIRE-A: The Shadow Pipeline
5. What You Get
6. Deployment Guide
7. Maintenance Requirements
8. Locked Design Decisions
9. What the Recipient Provides

CLAIRE is not a product. It is an architecture that requires deliberate configuration and weekly human maintenance. The value compounds over time as the pipeline learns which signal sources align with your actual friction patterns. Week one is noise reduction. Month three is a tuned signal filter with an eval history.

1. What is CLAIRE?

CLAIRE -- Claude Loop for Adaptive Intelligence from Real-world Evidence -- is a weekly automated pipeline that mines practitioner community signal from Reddit, HackerNews, and dev.to, filters it against your personal friction log, and synthesizes actionable candidates for improving how you work with Claude. It does not improve Claude's model weights. It improves your configuration: the memory edits, profile directives, skill definitions, and behavioral patterns that shape how Claude responds to you specifically.

The pipeline runs three models in sequence: Haiku for fast triage and noise filtering, Sonnet for synthesis and candidate generation, and Opus for the shadow decision engine (CLAIRE-A). Each weekly run produces a structured digest you review before anything touches your live configuration. Every change requires a human-written hypothesis. Every applied change enters an eval loop with a 14-21 day observation window.

The name is accurate. It is a loop. Community signal in, configuration candidates out, human review gate, applied change, friction observation, next cycle. Over time the pipeline develops a reliability ledger for its own signal sources and a scored history of which hypotheses held. That history is what CLAIRE-A uses to make increasingly calibrated shadow decisions.

2. The Problem It Solves

Claude's behavior is highly configurable -- memory edits, custom instructions, skills, and interaction patterns all shape output quality substantially. The problem is that most users configure once at setup and never revisit it. Meanwhile, the model improves, community practitioners discover new techniques, and your own working patterns evolve. Configuration drift is silent: you stop noticing friction because you have normalized workarounds.

At the same time, there is a constant low-level signal stream on platforms like Reddit and HackerNews where practitioners share what actually works -- specific prompt patterns, memory structures, skill designs, model-version

behavioral observations. This signal is buried in noise, arrives asynchronously, and requires manual synthesis to be useful. Most of it never reaches the people who would benefit from it.

CLAIRE's core claim: A structured weekly review of community signal, filtered against your own friction patterns and applied through a disciplined eval loop, produces measurably better Claude performance over time than static configuration or unstructured ad-hoc adjustments.

CLAIRE addresses both sides: it automates the signal collection and initial filtering, and it enforces the eval discipline that makes changes traceable and reversible. The friction log is the critical human input -- without it, the cross-reference gate has no ground truth and all signal scores medium regardless of personal relevance.

3. Pipeline Architecture

The pipeline runs five stages in sequence, orchestrated by a PowerShell wrapper (claire_weekly.ps1) scheduled via Windows Task Scheduler. Each stage exits cleanly on failure; the wrapper checks exit codes and aborts downstream stages rather than propagating corrupt state.

01 INGEST	claire_ingest.py Pulls posts from Reddit (PRAW), HackerNews (Algolia API), and dev.to. Applies a noise prefilter (score < 5 AND comments < 2 dropped). Outputs raw_posts.json. Runs with --source all flag.
02 TRIAGE	claire_triage.py Haiku classifies each post into Track A (direct config signal), Track B (skill build candidate), or Track C (technique worth testing). LOW confidence items archive. Requires 3+ corroborating posts for Track A promotion.
03 SYNT HESIZE	claire_synthesize.py Sonnet processes each track in parallel (ThreadPoolExecutor). Track A: memory/profile change candidates with fingerprints and change_target. Track B: SKILL.md skeletons. Track C: technique summaries. Batched by signal_type, max 50 posts per call.
04 OUT PUT	claire_output.py Assembles the weekly digest .docx. Six sections: executive summary, Track A candidates, Track B skill drafts, Track C techniques, cost log, and CLAIRE-A shadow decisions. Delivered to output/.
05 CLAI RE-A	claire_a_*.py Shadow pipeline. Assembler builds engine payload from Track A candidates and change_log history. Opus makes shadow decisions with hypotheses. Sonnet scores against reliability ledger. Writes nothing to live config. Human reviews Section 6 of digest before any live application.

Track Routing

Triage routes each post to one of three tracks. Track A contains direct configuration signal -- posts that suggest a specific memory edit, profile change, or behavioral adjustment. Track B contains skill build candidates -- observed capability gaps or community-shared techniques that warrant a new SKILL.md. Track C contains techniques worth

testing but not yet ready to apply. LOW confidence posts archive for quarterly review. The evidence threshold for Track A promotion is three corroborating posts minimum -- a single post, however compelling, does not qualify.

Cost Profile

A full weekly run across approximately 500 ingested posts costs in the range of \$0.60-\$0.70 USD in Anthropic API fees at current pricing. Haiku triage is the volume stage and remains cheap. Sonnet synthesis and Opus decision engine runs are the cost-significant stages. A cost_log.json tracks per-run spending. Track A batching (max 50 posts per synthesis call) is the primary cost control lever.

4. CLAIRE-A: The Shadow Pipeline

CLAIRE-A is a parallel decision layer that runs after main synthesis. It reads Track A candidates, the change log, session history, and source reliability data, then uses Opus to make shadow configuration decisions -- complete with hypotheses, confidence ratings, and reasoning. These decisions appear in Section 6 of the weekly digest for human review.

CLAIRE-A writes nothing to live configuration. It is shadow-only by architecture. The safety property is enforced at the code level: the runner writes only to claire_a_decisions_[timestamp].json and the digest. It has no write path to memory, config.json, or any skill file. Human authorization is required for every applied change.

Graduation Criteria

CLAIRE-A operates in permanent shadow mode until it demonstrates sufficient calibration across six consecutive eval runs. Graduation requires: 80% or higher agreement rate with human decisions, 10+ scored hypotheses in the reliability ledger, and zero escalations in the last three runs. Current status as of Build 7: one of six consecutive runs logged, seven of ten hypotheses scored, zero escalations. Graduation is not imminent and that is by design.

Design intent: CLAIRE-A's value at current maturity is not its decisions. It is the structured reasoning it produces alongside those decisions -- the hypotheses, confidence ratings, and source attributions that make the human review faster and better calibrated. Treat Section 6 of the digest as a prepared briefing, not a decision record.

5. What You Get

Each weekly run produces a structured digest in the output/ directory. The digest is a .docx file with six sections:

Section	Title	Contents
Section 1	Executive Summary	High-signal summary of this week's candidates, estimated cost, source breakdown.
Section 2	Track A Candidates	Memory edits and profile changes ready for human review. Each includes the source post(s), signal type, confidence rating, fingerprint, and change_target field.
Section 3	Track B Skill Drafts	SKILL.md skeletons for new skills identified from community signal. Ready to review and refine before packaging.

Section 4	Track C Techniques	Prompt patterns, interaction techniques, and model-behavior observations worth testing. Not yet ready to apply as configuration changes.
Section 5	Cost Log	Per-stage API cost breakdown for this run and rolling history.
Section 6	CLAIRE-A Shadow Decisions	Opus decision engine output: shadow recommendations with hypotheses, confidence ratings, and source attribution. Human review only -- not applied.

What Accumulates Over Time

Beyond the weekly digest, the pipeline accumulates structured artifacts that compound in value: `change_log.json` with applied changes and eval notes builds into a searchable configuration history; the source reliability ledger scores each ingest source on prediction accuracy; the session history enables CLAIRE-A to identify recurring signal patterns across runs. After three months of consistent operation, these artifacts form a documented configuration evolution record that can drive quarterly eval sessions.

6. Deployment Guide

Prerequisites

Decision	Value
Python 3.10+	Windows or macOS. Windows path examples use PowerShell.
Anthropic API key	Required. All three pipeline models (Haiku, Sonnet, Opus) bill against this key. Estimated \$0.60-\$0.70 per weekly run.
Reddit API credentials	Free. Register a 'script' type app at reddit.com/prefs/apps. Takes 10 minutes. Read-only access -- no posting permissions needed.
HackerNews	No credentials required. Uses the public Algolia search API.
dev.to	No credentials required. Uses the public dev.to API.
Git (optional)	Recommended for tracking config changes over time.
Windows Task Scheduler	For automated weekly runs. The .bat launcher handles venv activation and log piping. Linux/macOS can substitute cron + bash.

Setup Steps

- 1
- Clone or extract the repository**

Place in a project directory of your choice. The path is arbitrary but should not contain spaces if using the .bat launcher on Windows.

-
- 2 Create and activate a virtual environment**
python -m venv .env | Windows: .env\Scripts\activate.bat | macOS/Linux: source .env/bin/activate | Then: python -m pip install -r requirements.txt

 - 3 Configure credentials**
Copy .env.example to .env. Add your ANTHROPIC_API_KEY, REDDIT_CLIENT_ID, REDDIT_CLIENT_SECRET, and REDDIT_USER_AGENT. Never commit this file.

 - 4 Configure config.json**
Copy config.template.json to config.json. Set your target subreddits, HackerNews search terms, signal_map, and exclude_keywords. Start conservative -- you can widen sources after the first few runs.

 - 5 Write your profile_intent_summary.txt**
This is the most important configuration step. Describe who you are, what you use Claude for, what quality means in your context, and what kinds of changes you want CLAIRE to surface. One to three paragraphs. Be specific. See Section 9 for guidance on what to include.

 - 6 Initialize data files**
Create empty data/change_log.json and data/friction_log.txt using the templates provided. These are human-maintained -- the pipeline reads them but never overwrites them.

 - 7 Run the first ingest**
python claire_ingest.py --source all | Expect 300-600 posts. Review ingest.log for errors. If Reddit credentials fail, verify the REDDIT_USER_AGENT format: 'CLAIRE/0.1 by /u/YOUR_USERNAME'

 - 8 Run triage**
python claire_triage.py | First run classifies the full corpus. Subsequent runs are incremental. Review triage.log for track distribution. Expect roughly 25-35% Track A, 1-5% Track B, 3-8% Track C.

 - 9 Run synthesis and output**
python claire_synthesize.py | Then: python claire_output.py | The digest appears in output/. Review it before running CLAIRE-A.

 - 10 Run CLAIRE-A (optional first run)**
python claire_a_assembler.py | python claire_a_runner.py | python claire_a_scorer.py | Section 6 of the digest will populate on next output run. Shadow decisions only -- nothing applied.

 - 11 Schedule the pipeline**
Edit claire_weekly.ps1 to match your project path. Create a Windows Task Scheduler task pointing to the .bat launcher. Set to run weekly, 'Run whether user is logged on or not', highest privileges.

7. Maintenance Requirements

CLAIRE requires two human-maintained files. Without consistent maintenance of both, the pipeline degrades predictably and measurably.

Weekly (10 minutes)

Decision	Value
friction_log.txt	Add 2-4 entries describing friction you experienced with Claude this week. Format: date context what happened what you wanted instead. This is the cross-reference gate's ground truth. Without it, all signal scores MEDIUM regardless of relevance. The bias-correction layer collapses.
change_log.json	For any candidate you decide to apply from the digest: add an entry with your hypothesis before applying the change. One sentence: what behavior do you expect to change, in what context, observable how? If you cannot write this sentence, do not apply the change.

Quarterly (30-60 minutes)

Feed change_log.json and friction_log.txt to Claude and ask for an eval report. Which applied hypotheses held? Which produced friction or neutral results? Changes with consistent positive eval notes stay. Neutral changes get reviewed. Changes with friction get reverted and the source signal gets flagged low-quality in the reliability ledger. Review the archive.json for any LOW-confidence posts that have since been corroborated by new signal.

What degrades without maintenance: Without friction_log.txt updates, CLAIRE becomes a Reddit-to-config pipeline with no personal filtering -- it will surface community best practices that may be actively wrong for your use case. Without change_log.json hypotheses, the quarterly eval has nothing to measure against and you lose the eval loop entirely. The 10-minute weekly investment is the mechanism that makes everything else meaningful.

8. Locked Design Decisions

These decisions are locked in the reference implementation. They reflect tradeoffs established through iterative development and should not be changed without a documented hypothesis and at least one full eval cycle.

Decision	Value
Triage model	claude-haiku-4-5-20251001 -- volume stage, cost-optimized
Synthesis model	claude-sonnet-4-6 -- quality/cost balance for candidate generation
Decision engine model	claude-opus-4-5 -- shadow decisions only, highest reasoning quality
Eval scoring model	claude-sonnet-4-6 -- scores CLAIRE-A hypotheses against friction log

Evidence threshold	3 corroborating posts minimum for Track A promotion
Noise prefilter	Score < 5 AND comments < 2 dropped before triage
CLAIRE-A mode	Shadow only -- zero write access to live config files
Track A batch ceiling	50 posts per synthesis call, batched by signal_type
CLAIRE-A batch ceiling	15 candidates per decision engine run
Hypothesis authorship	Human-written for applied changes; Opus-written for shadow decisions
Eval windows	14 days for format/behavior changes; 21 days for memory/behavioral
Config injection depth	Partial -- profile_intent_summary.txt + memory list injected into Track A/B
Developer persona	Filtered out at triage -- developer-audience posts excluded
Ingest sources	Reddit (native + search) + HackerNews (Algolia) + dev.to (tags)

9. What the Recipient Provides

The repository ships as architecture -- the engine without the fuel. The following must be supplied by the person deploying CLAIRE. None of it is optional; each item is load-bearing.

Decision	Value
API credentials	Anthropic API key, Reddit client_id and secret, Reddit user agent string. Into .env -- never into any Python file.
config.json	Subreddits to monitor, HackerNews search terms, signal_map (maps signal types to change targets), exclude_keywords (keeps Opus-bait out of Track A), dev.to tag list. Start narrow; expand after first three runs.
profile_intent_summary.txt	Who you are, what you use Claude for, what quality means in your context, what kinds of change candidates are useful vs. noise for you specifically. This is injected into every Track A and B synthesis call. It is the personalization layer. Generic text produces generic candidates.
friction_log.txt	Ongoing weekly log of friction experienced with Claude. The cross-reference gate compares incoming signal against this file to score relevance. Without personal friction data, signal scoring is essentially random.

change_log.json	Starts empty. Populated manually as you apply candidates from the digest. The hypothesis field is mandatory -- it is what the eval loop measures against.
Observation discipline	After applying a change, you must observe whether the hypothesis held over the eval window and add eval notes to change_log.json. The pipeline cannot do this for you. This is the closing mechanism of the feedback loop.

A Note on profile_intent_summary.txt

This file is the soul of the configuration. It tells the synthesis models what to look for, what to elevate, and what to suppress. A vague intent summary produces candidates that reflect community best practices for the median Claude user -- which is probably not you. Specificity is the mechanism. Include your primary use cases, your output quality bar, the environments where your work enters formal review processes, and any secondary use cases that should only generate candidates on direct signal evidence (not adjacent discussion). One to three paragraphs. Revise it after the first quarterly eval.

CLAIRE was designed for a senior technology executive in a regulated environment where AI output quality directly affects organizational decisions and formal document review processes. The architecture is generic; the configuration is not. The pipeline will faithfully optimize for whatever profile_intent_summary.txt says you care about. Write that file with the same care you would apply to any system prompt that shapes high-stakes outputs.